BTF Trace Viewer

Version 1.4.0 — Desktop and Web

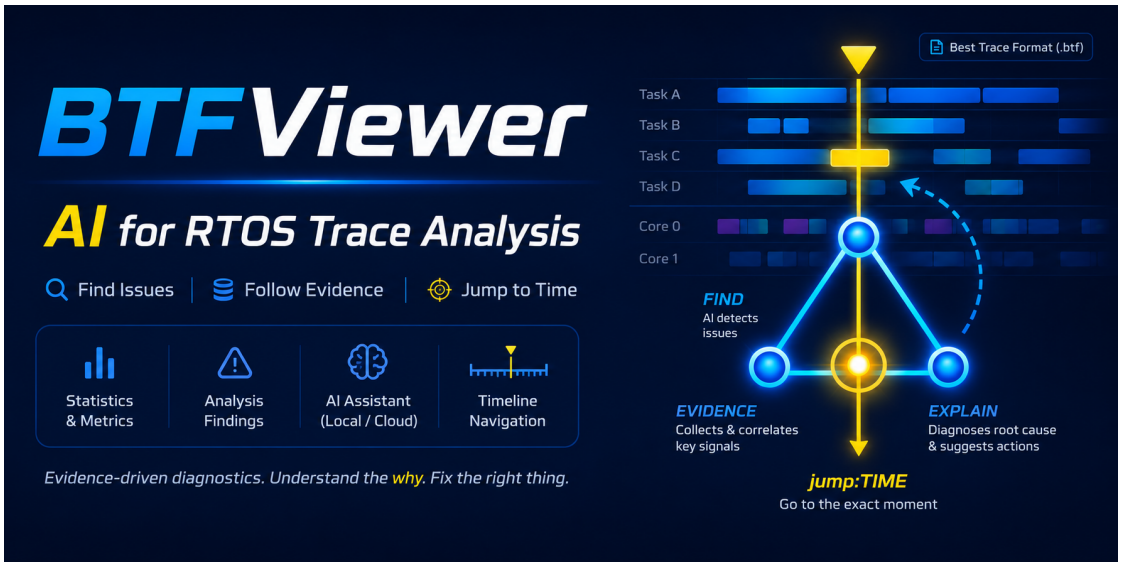

Figure 1: BTFViewer AI-assisted analysis

BTFViewer turns BTF traces into verifiable evidence about multicore scheduling. It provides trace-based statistics, before-and-after comparison, reports for continuous integration (CI), and optional AI-assisted investigation.

BTFViewer analyzes context-switch events recorded by a real-time operating system (RTOS) in **Best Trace Format** (`.btf`). You can use the Desktop or Web application after a trace has been captured. BTFViewer complements low-level debuggers and target-side trace recorders. It does not read source code or ELF files, and it does not simulate the scheduler. All results come from events in the trace.

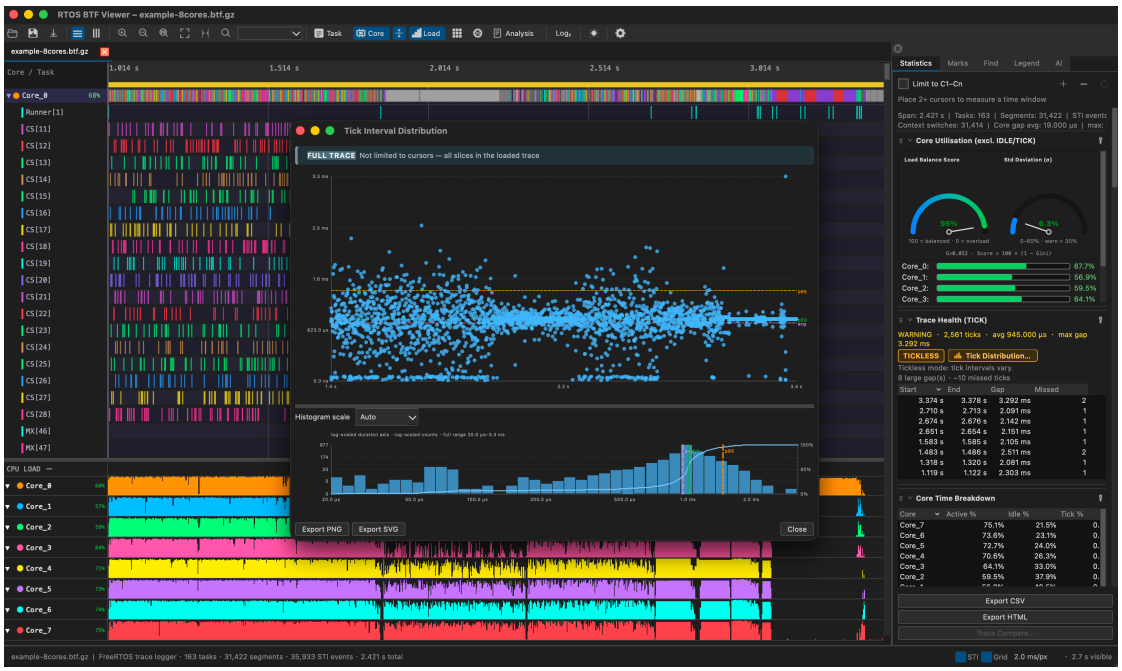

Figure 2: BTFViewer

Try the live demo

Why BTFViewer?

BTFViewer helps you move from a timing symptom to evidence that others can review and reproduce.

- **Evidence you can verify:** Statistics and Analysis Findings are calculated from recorded BTF/STI events. Metric definitions, limitations, and timeline locations help you check each result against the trace.
- **Multicore behavior made explicit:** Dedicated diagnostics reveal load imbalance, concurrent core activity, task affinity, migration frequency, ping-pong movement, and hot core-to-core migration paths—not just which task ran when.
- **Comparison built into the workflow:** Open multiple traces, measure before-and-after deltas and distribution changes, save baselines, and use the Desktop CLI as a regression gate in automated tests or CI.

- **AI based on measured data:** The optional AI Assistant can explain findings, investigate possible causes, verify evidence, and summarize trace comparisons. Statistics and the timeline remain the source of truth. A local OpenAI-compatible service can keep the extracted evidence on your machine.
- **Easy to run and share:** Desktop and Web use the same analysis workflow. The standalone Web build and bilingual guided demo make it easy to review or demonstrate a trace without a specialized analysis installation.

Features

Area	Capabilities
Timeline and measurement	Task or CPU-core views, horizontal or vertical layouts, zoom, pan, search, cursors, bookmarks, and annotations
Statistics and diagnostics	Utilisation, execution, blocking, dispatch latency, jitter, preemption, migration, mutex, semaphore, queue, deadline, and anomaly analysis
Trace comparison	Multi-tab sessions, before-and-after deltas, distribution comparisons, saved baselines, and experiment validation
AI Assistant	Finding triage, focused investigation, evidence verification, what-if analysis, and comparison explanations
Export and automation	PNG, SVG, HTML reports, Perfetto, selected BTF ranges, and Desktop CLI support for scripts and CI
Learning and sharing	Standalone Web application and an 8-core guided demo with English or Traditional Chinese narration

Documentation

Document	Purpose
README.md	Install BTFViewer and learn its main features
WORKFLOWS.md	Follow step-by-step investigation procedures
STATISTICS.md	Understand metric definitions, formulas, and interpretation
AI.md	Configure and use AI-assisted investigation
demos/README.md	Create and maintain guided demos

If you are new to BTFViewer, begin with Quick start, then follow the Basic analysis workflow. Refer to the other documents for detailed procedures and metric definitions.

Quick start

Desktop application

Requirements:

- Python 3.8 or later. Python 3.9 or later is recommended.
- PySide6 6.4 or later. The command below installs it with the other dependencies.

```
cd BTFViewer
pip install -r requirements.txt
python builds/btf_viewer.py trace.btf
```

Replace `trace.btf` with the path to the trace file. If no file is specified, BTFViewer restores the previous session. Files can also be opened through **File → Open**, **File → Open Recent**, or drag and drop.

Web application

Open the standalone file in a modern browser:

```
open BTFViewer/builds/btf_viewer.html
```

You can also double-click the file or use the hosted demo. A local server is usually not required.

To rebuild the Web application:

```
cd BTFViewer
make web
```

Supported files

Format	Description
`.btf`	Best Trace Format
`.btf.gz`, `.gz`	Gzip-compressed trace
`.btf.bz2`, `.bz2`	Bzip2-compressed trace
`.btf.zip`, `.zip`	Archive containing one or more BTF traces; each trace opens in a separate tab
`.xml`	Demo script used by the Web application
`.xtf`	Portable demo package containing a script, trace, and optional narration

Sample traces are available in `tracedata/`, including `example-2cores.btf.gz`.

Guided demo

The `demo_8cores` package presents the main workflow with an 8-core trace and spoken instructions. Run it before opening an application trace to become familiar with the interface and analysis sequence.

Run the demo

Desktop:

```
cd BTFViewer
make demo
make demo DEMO_LANG=zh-tw
```

You can also open either package directly:

```
python builds/btf_viewer.py demos/demo_8cores/demo_8cores.xml
python builds/btf_viewer.py builds/demo_8cores.xtf
```

Web:

- Select **Demo** on the toolbar to load the bundled tour.
- Open or drag `demo_8cores.xtf` into the viewer.
- Open the demo XML file and select its package folder when requested.

English is the default narration language. Select another language from the demo bar **Voice** menu. Press **Space** to pause or resume. Press **Esc** twice within 2.5 seconds to stop.

Build a portable demo package

```
make demo-pack
make demo-pack DEMO_LANGS=en,zh-tw
```

The generated `builds/demo_8cores.xtf` contains the script, trace, and selected voice files. Open it in either application. See demos/README.md for instructions on creating, recording, and maintaining demos.

Viewer controls

The viewer uses one interface, one set of controls, and one analysis workflow. During an investigation, check the active trace, **Scope**, **Filters**, View Mode, **Selection**, and **Highlight** before interpreting a result.

The main window

The Desktop and Web builds share one layout.

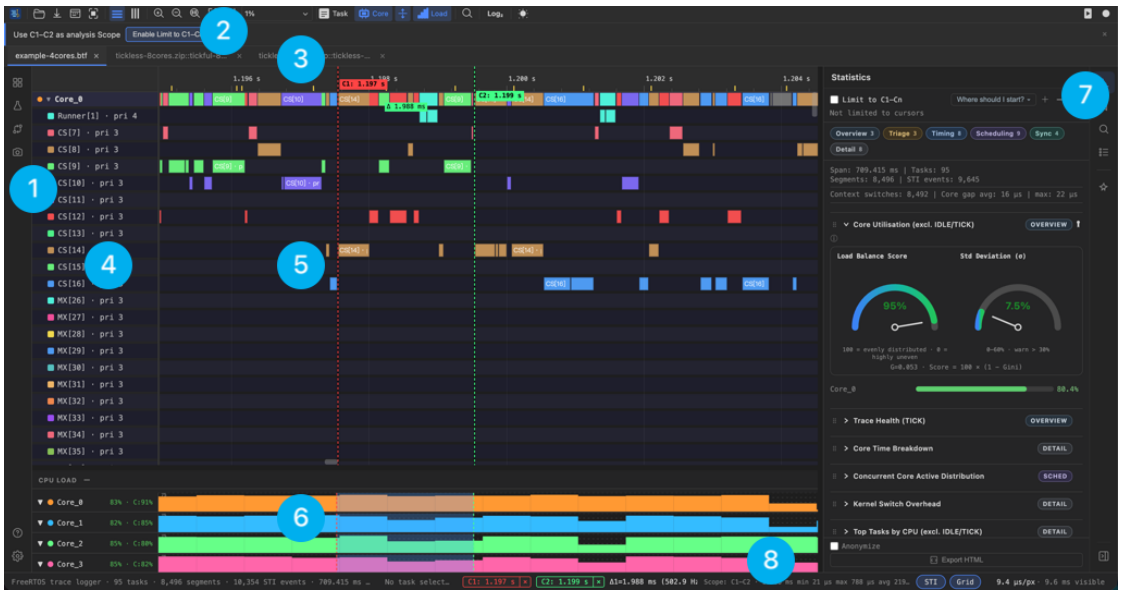

Figure 3: BTFViewer main window

#	Region	What it does
1	**Activity rail**	Opens the tools that work alongside the timeline — **Migration heatmap**, **Analysis Findings**, and the **Snapshot editor** — with **Help** and **Settings** at the bottom.
2	**Toolbar**	Grouped controls — Open, layout, zoom, View Mode, **Load**, theme, demo, record. Hover an icon for its name and shortcut; a narrow window folds groups into **More (⋯)**. See Main controls.
3	**Trace tabs**	One tab per open trace. Each tab keeps its own Scope, Filters, View Mode, zoom, cursors, and marks.
4	**Legend / task list**	Colour key for the rows. Hover an entry for **Highlight**, click it for **Selection**; in Core View the **Cores** checkboxes act as the Core Filter.

#	Region	What it does
5	**Timeline and time ruler**	The scaled event view. `Ctrl`+scroll zooms toward the pointer, drag to pan, click to place a cursor, and hover a segment for task, core, start/end, and duration. The small framed box at the lower right is a whole-capture overview.
6	**CPU load graph**	Utilisation below the timeline; toggle it with **Load** and drag the divider to resize. With a task selected, Task View can show that task's per-core utilisation here.
7	**Right panel**	The icon rail switches between **Statistics**, **Marks**, **Find**, **Legend**, and the **AI Assistant**. Statistics (shown) holds the **Limit to C1–Cn** toggle, the investigation-category filter, the summary, the collapsible metric sections, and **Anonymize** + **Export HTML** in its footer.
8	**Status bar**	Active-trace summary, current **Selection**, **Scope**, **Filtered:** state, zoom (µs/px and visible span), and Find `k of N` results.

Investigation terminology

Term	Meaning
Full Trace	No cursor-defined time window; analysis uses the whole capture
Scope	The time range used for analysis (**Full Trace** or **C1–Cn · duration**)
Filter	A data subset inside the Scope (Task, Core, or Migration). Clears with **×** or **Clear All**
Selection	The task or object kept active for inspection. It does not change the analyzed data by itself

Term	Meaning
Highlight	Temporary visual emphasis, such as hovering over a Legend item. It does not change the analyzed data
Fit Trace	Zoom the viewport to the complete capture (`Ctrl+0` / F)
Fit Cursors	Zoom the viewport to the earliest–latest cursor span (`Ctrl+R`)
Baseline / Candidate	Trace A and Trace B in Trace Compare
Regressed / Improved	Compare verdicts when a metric moves against or toward the baseline

Selection and Highlight never silently become a Filter. View Mode (**Task** / **Core**) is independent of Selection, Highlight, and Filter.

The status bar shows the current Scope, active Filters, and Zoom. Statistics also shows **Filtered:** when a Filter limits the analyzed data. Filters are stored separately for each analysis tab.

Main controls

The toolbar groups related controls. Hover over an icon to see its name and shortcut.

Group	Controls	Purpose
Open	**Open**	Open one or more BTF traces, a demo XML file, or an `.xtf` demo package
Capture and export	**Snapshot editor**, **Save SVG**, **Export Perfetto**, **Save cursor range as BTF**	Capture the current view or export trace data. Saving a BTF range requires at least two cursors

Group	Controls	Purpose
Layout	**Horizontal** / **Vertical**	Run the time axis from left to right or from top to bottom
Zoom	**Zoom in** / **Zoom out**, **1:1**, **Fit Trace**, **Fit Cursors**, zoom preset	Change the visible time range or select a fixed time-per-pixel scale
View	**Task** / **Core**, expand or collapse all cores, **Load**	Choose the timeline grouping and show or hide the CPU-load graph
Investigation	**Find**, **Migration & Corridor Inspector**, **Analysis**, **Compare**	Locate evidence, inspect multicore movement, open findings, or compare a Baseline and Candidate
Conditional controls	**All tasks**, **Log₂**	Clear an active Migration Filter, or change an expanded STI waveform between linear and logarithmic scaling
Display	Light/dark theme	Switch the viewer theme without opening Settings
Support	**C1–Cn**, **Demo**, **Record**, **Settings**, **Help**	Show whether Statistics is limited to C1–Cn (colour on vs off), load the bundled demo, record the current tab as WebM, configure the viewer, or open shortcuts and help

Select the application icon to open **About**. When the window is too narrow for every group, the toolbar moves groups into **More (⋯)** without changing their functions.

Task and core views

View	Use
Task View	Follow each task across all cores
Core View	See which task ran on each core; cores can be expanded or collapsed

Task View is useful for checking execution and migration. Core View is useful for checking utilization, idle periods, and load distribution. Switching View Mode preserves Timeline position, Zoom, Cursors, and Scope.

In **Core View**, the Legend **Cores** checklist is the **Core Filter**: unchecked cores are removed from Timeline Core View rows, the CPU Load graph, the status-bar chip, Statistics **Filtered:** state, and AI context.

Zoom, Selection, and Highlight

- Use the mouse wheel to pan. Hold **Ctrl** while scrolling to zoom.
- Hold **Shift** while scrolling to change the pan axis.
- Use a trackpad pinch gesture to zoom on macOS.
- Middle-drag across the timeline to zoom into a selected time range.
- Select **Fit Trace** or press `Ctrl+0` / F to show the complete capture. **Fit Cursors** / `Ctrl+R` fits the time between the earliest and latest cursor (C1–Cn).
- Select **1:1** to return to the configured zoom density.
- **Hover** a Legend entry or timeline segment for **Highlight** (transient). **Click** a task label or Legend entry for **Selection** (persistent). These do not apply a Filter.
- Hover over a timeline segment to view task, core, start/end time, and duration.

CPU load

Select **Load** to display the utilization chart below the timeline. Drag the divider to resize it.

When a task is the current **Selection**, Task View can show its utilization on each core. Use this display to check whether the work is distributed as expected or moves between cores too often.

Cursors and Scope

Cursors mark timestamps and can define a measurement Scope. BTFViewer supports four cursors by default; you can change the limit in **Settings**.

Action	Method
Place a cursor	Click the timeline, or press `C` to place one at the viewport center
Move a cursor	Drag its line
Remove a cursor	Click near the line or use the context menu
Clear all cursors	Press `Shift+C` or Shift-right-click
Snap to an event boundary	Shift-click

Place at least two cursors around the period you want to inspect. Enable **Limit to C1–Cn** in Statistics, from the toolbar **C1–Cn** chip, or from the command palette so calculations use the earliest–latest span. The chip uses the Scope colour when Limit is on and a muted colour when it is off. The status-bar Scope line updates immediately. **Fit Cursors** (`Ctrl+R`) displays only this period. **Save cursor range as BTF** exports it as a smaller trace.

Marks, bookmarks, annotations, and Find

Tool	Purpose
Cursor	Temporary measurement / investigation point
Bookmark	Saved location for later return
Annotation	Human-written note tied to a Trace timestamp
Find	Locate tasks, migrations, STI events, intervals, lifecycle events, and synchronization objects

The Marks panel lists cursors, the cursor range, bookmarks, and annotations. Use **Export Marks** / **Import Marks** or **Export Session** / **Import Session** to share them.

Use `Ctrl+F` to open Find. Status shows **`k of N matches`**. Use Previous/Next, F3, and `Shift+F3` to move between results without changing Scope or Filters. Match Mode details live in tooltips. Right-click the timeline for cursor and mark actions.

Multiple traces

Each trace opens in a separate tab and keeps its own zoom, cursors, marks, and Filters.

- `Ctrl+Tab`: next tab
- `Ctrl+Shift+Tab`: previous tab
- `Ctrl+W`: close the current tab

Desktop can reopen files from their original paths. Web restoration depends on browser storage and may be unavailable in private browsing mode or after site data is cleared.

Basic analysis workflow

For an initial review, use the following sequence:

1. Open the trace and select **Fit Trace** to view its complete duration.
2. Select **Load** and check whether all cores carry a reasonable share of the work.
3. Open **Analysis** and review the highest-severity findings (Triage).
4. Select **Investigate** on a finding to open the relevant Statistics section and apply the finding's recommended Scope (C1–C2). Filters are preserved.
5. Select **Show on timeline** to center the timeline on the relevant event. This action does not change the Scope or Filters.
6. Use **Back** / **Forward** in the evidence inspector to revisit prior timeline jumps without changing Scope or Filters.
7. Select a high value or outlier to jump to the corresponding timeline event.
8. Place cursors around the affected period, confirm **Scope: C1–Cn** in the status bar, and enable **Limit to C1–Cn**.
9. Inspect the task, core, preemption, blocking, synchronization, or migration details.
10. If needed, ask the AI Assistant to explain or verify the measured evidence.

Start with measured evidence instead of an assumed cause. Confirm the behavior in the timeline and Statistics before drawing a conclusion. See WORKFLOWS.md for detailed procedures.

Analysis and Statistics

BTFViewer calculates all results from recorded BTF events. It does not inspect source code or ELF files, simulate an RTOS scheduler, or estimate data that is not present in the trace.

Choose the first check

Symptom	Start here	Check next
Unknown issue	**Analysis Findings**	Statistics section named by the finding
Tick jitter or tickless behavior	**Trace Health (TICK)**	Execution-time outliers
Uneven SMP load	**Core Utilisation**	Concurrent active cores, then migrations
High scheduler cost	**Kernel Switch Overhead**	Core time breakdown
Slow task execution	**Execution Time**	Preemption and mutex activity
Long wait time	**Blocking Time**	Mutex owner and preemption activity
Ready-to-run delay	**Dispatch Latency**	Blocking and preemption
Priority inversion	**Priority Inheritance**	Mutex pairing and blocking

Symptom	Start here	Check next
Frequent core movement	**Core Migrations**	Load balance, Migration & Corridor Inspector, and synchronization handoff suspects
Lock or queue issue	**Mutex / Semaphore / Queue**	Blocking and migrations

Detailed metric definitions and formulas are in STATISTICS.md.

Analysis Findings

Select **Analysis** to open findings for the current **Scope** (**Full Trace** or **C1–Cn**). The timeline remains available while the Findings window is open. Findings may include load imbalance, long execution or blocking time, priority inversion, frequent core migration, missed deadlines, tick problems, and synchronization objects that move between cores.

Each finding includes:

- A clear **Severity** and problem-oriented title.
- The most relevant supporting metric.
- An **Evidence** line that separates the measured observation from its interpretation.
- **Investigate** — apply the suggested Scope and open the relevant Statistics section without using AI.
- **Show Evidence** — move the timeline to the supporting event without changing the Scope or Filters.
- Optional AI actions when the AI Assistant is configured.

Treat findings as leads, not confirmed root causes. Apply cursors when a finding recommends a useful window, then recheck Statistics inside that Scope.

For multi-core traces with measurable utilization, the core-balance finding reports a **Load Balance Score** with supporting distribution values. A high score means work is distributed more evenly. Review the timeline and migration data before deciding whether the distribution is suitable for your workload.

Open the Findings window from the toolbar **Investigation** group or the activity rail's **Analysis findings** button.

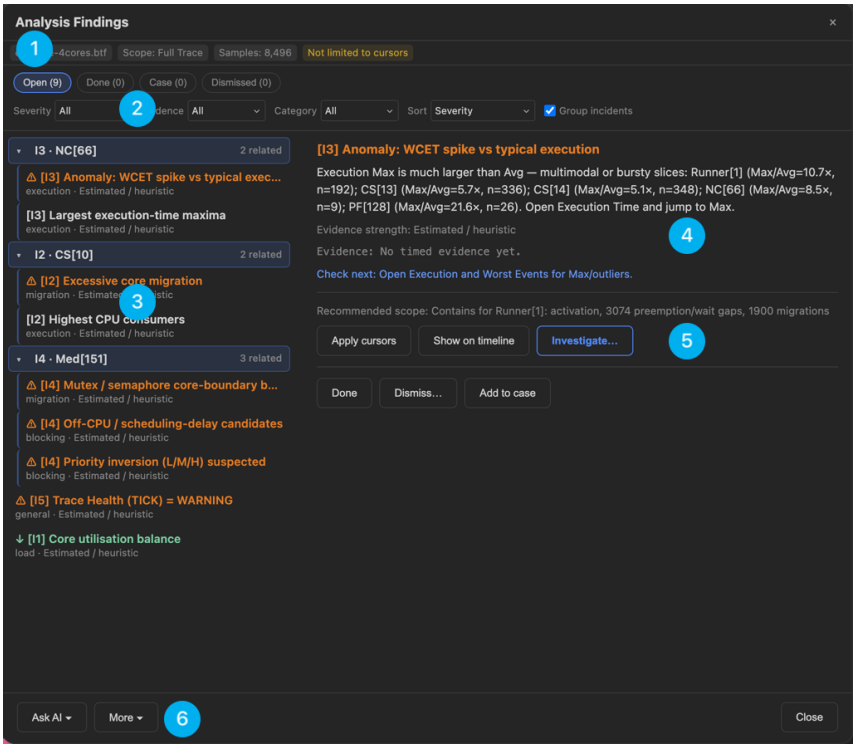

Figure 4: Analysis Findings

#	Region	What it does
1	**Context and triage state**	The active trace, current **Scope**, and sample count, plus the **Open** / **Done** / **Case** / **Dismissed** tabs that follow each finding through triage.
2	**Filters and sort**	Narrow by **Severity**, **Evidence** strength, or **Category**, change the **Sort**, and toggle **Group incidents** to fold repeats of one issue together.
3	**Findings list**	Severity-ranked and grouped by the affected object; each entry shows its category and whether the evidence is `Estimated / heuristic` or measured.

#	Region	What it does
4	**Detail pane**	The selected finding in full — observation vs interpretation, evidence strength, the exact evidence, a **Check next** pointer, and the recommended Scope.
5	**Finding actions**	**Apply cursors** / **Show on timeline** / **Investigate…** move to the evidence; **Done** / **Dismiss…** / **Add to case** record the triage outcome.
6	**Footer**	**Ask AI** about the selected finding, plus **More** export and case options.

Reading Max, p95, and p99

- **Max** is the largest measured value. Use it to locate the worst observed event.
- **p95** is the value that 95% of samples do not exceed. It shows behavior during the slower part of normal operation without being dominated by one rare event.
- **p99** is the value that 99% of samples do not exceed. Use it to identify severe but recurring latency that an average may hide.

p95 is important because an average alone does not describe real-time performance. A good average can still hide repeated slow events. Compare p95 with p99 and Max to distinguish typical tail latency from less frequent extremes.

Core migration checks

Check load balance before interpreting migration counts. An SMP scheduler may move tasks to idle cores to distribute the workload, so some migration is expected.

After confirming load balance, check whether a task moves between cores more often than needed. Frequent migration can increase L1 cache misses because a task may not find its recent data in the new core's cache. On Xtensa processors, migration can also reduce the benefit of lazy context switching. The system may need to save coprocessor registers when a task moves to another core, which increases context-switch overhead.

Review Task View, per-core load, **Core Migrations**, and the **Migration & Corridor Inspector** together. The Inspector shows **Core path**, **migration heatmap**, and **Topology** side by side. Click the heatmap to open **Path info** in the right column. **Analysis Scope** defaults to **Follow zoom** (Fit is Full Trace; a zoomed-in window is Viewport). You can lock Full Trace or Viewport, or choose Cursor C1–Cn. A high migration count is significant when it coincides with poor cache behavior, greater context-switch overhead, higher latency, or unstable load distribution. Handoff correlation is a heuristic, not a measured cache-line transfer.

Open the Inspector from the toolbar **Investigation** group or the activity rail's **Migration heatmap** button.

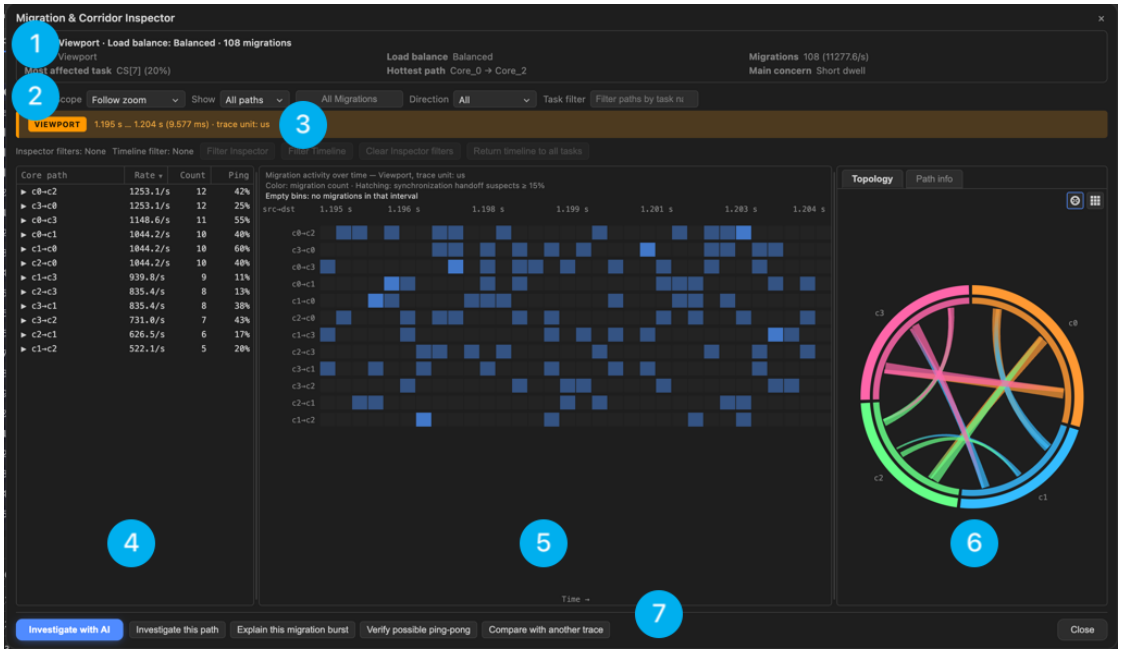

Figure 5: Migration & Corridor Inspector

#	Region	What it does
1	**Verdict strip**	A one-line read plus the supporting facts: Scope, Most affected task, Load balance, Hottest path, total Migrations (with rate), and the Main concern.

#	Region	What it does
2	**Toolbar**	**Analysis Scope** (Follow zoom / Full Trace / Viewport / Cursor C1–Cn), **Show** path depth, a toggle between **All Migrations** and bounce (ping-pong) paths only, **Direction**, and a **Task filter**.
3	**Scope banner**	The exact window and time unit the numbers cover, so a zoomed or cursor-limited view is never mistaken for the whole trace.
4	**Core-path tree**	Source → destination corridors ranked by rate, with Count, Ping (bounce share), Dwell, Handoff, Net, and Share. Expand a row (▶) for its per-task breakdown.
5	**Migration heatmap**	Migrations per time bin for each corridor — cell colour is the count, hatching marks intervals where synchronization-handoff suspects exceed the threshold, and empty bins are labelled. Click a cell to focus that corridor and interval.
6	**Topology / Path info**	A chord view of core-to-core flow; the **Path info** tab shows the selected corridor's detail. The buttons at the top right switch the layout.
7	**AI actions**	**Investigate with AI**, plus one-click prompts — Investigate this path, Explain this migration burst, Verify possible ping-pong, and Compare with another trace.

Comparing open traces

Compare is available when two or more traces are open. It summarizes differences in utilization, migrations, execution, blocking, response time, synchronization activity, and deadline misses.

The **Summary** tab identifies the **Baseline** (reference trace) and **Candidate** (new trace), counts regressions and improvements, and explains the overall result. Small

changes without practical engineering significance are omitted. Click a column header to sort any compare table; click again to reverse the order.

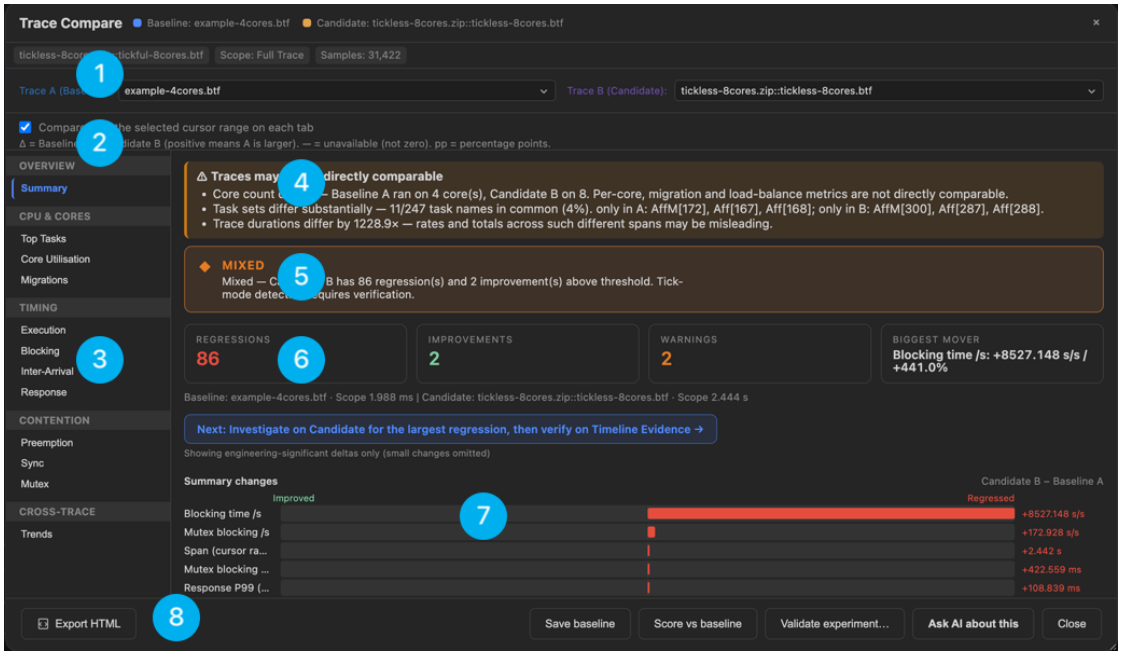

Figure 6: Trace Compare

#	Region	What it does
1	**Trace A / Trace B selectors**	Choose which open trace is the **Baseline (A)** and which is the **Candidate (B)**.
2	**Scope and Δ convention**	Optionally limit the comparison to each tab's cursor range; the note states that **Δ = Baseline A − Candidate B** (– means unavailable, pp = percentage points).
3	**Section rail**	Jump between compared areas — Summary, Top Tasks, Core Utilisation, Migrations, Execution, Blocking, Inter-Arrival, Response, Preemption, Sync, Mutex, and cross-trace Trends.

#	Region	What it does
4	**Comparability check**	Warns when the traces are not directly comparable — different core counts, or task sets that barely overlap — so per-core and load-balance deltas are read with care.
5	**Verdict banner**	The overall call (Improved / Regressed / Mixed / Unchanged) with the regression and improvement counts behind it.
6	**Summary cards**	Regressions, Improvements, Warnings, and the single **Biggest mover**.
7	**Change chart and metrics table**	Engineering-significant deltas as a diverging bar chart (improved ↔ regressed), with the full **All summary metrics** table (Metric / Baseline A / Candidate B / Change) below it.
8	**Footer**	**Export HTML**, **Save baseline** / **Score vs baseline**, **Validate experiment…**, or **Ask AI about this**.

This is an optional comparison tool. It is not required by the basic investigation workflow. When you use it, compare equivalent workload phases and measurement ranges.

AI Assistant

The optional AI Assistant explains Analysis Findings and Statistics measured by BTFViewer. It does not replace timeline verification or create measurements that are missing from the trace.

Open it from the right panel’s icon rail (**AI Assistant**).

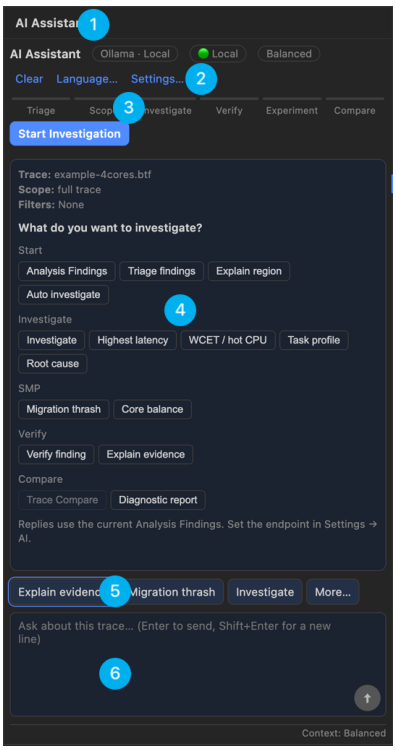

Figure 7: AI Assistant panel

#	Region	What it does
1	**Header**	The panel title, the provider / privacy chip (**Local** vs **Cloud**), and the active context level.
2	**Toolbar**	**Clear** the conversation, switch reply **Language…**, or open **Settings…** (model, endpoint, authentication, privacy).
3	**Guided investigation**	The **Triage → Scope → Investigate → Verify → Experiment → Compare** stepper and **Start Investigation**, which walks a structured workflow.
4	**Conversation and starter prompts**	The reply thread. Before the first message it shows the current Trace / Scope / Filters and grouped one-click prompts (Start, Investigate, SMP, Verify, Compare).

#	Region	What it does
5	**Quick-action templates**	Ready-made prompts for the current selection — **Investigate**, **Verify finding**, **Explain evidence**, and **More…**.
6	**Composer and context meter**	Type a question (Enter sends, Shift+Enter for a new line); the meter below shows the context level used for the request.

Recommended use:

1. Select a finding or define a time range with cursors.
2. Ask the AI Assistant to investigate or explain it.
3. Review the cited Statistics and timeline evidence.
4. Use **Verify with AI** to challenge the proposed cause.
5. If you make a change, capture a new trace and repeat the same scoped measurements.

Select text in a reply and right-click **Ask AI (preview…)** to send that snippet as a follow-up (enabled for two or more words). Replies can include `nextstep:{action}` on its own line (`nextstep:` in English; the action in the reply language); conversation **[Run]** sends that follow-up.

Available context levels are **Compact**, **Balanced**, and **Full Evidence**. Compact is fast triage; Balanced is the default investigation depth; Full Evidence adds relevant findings and investigation history. Confidence comes from evidence, not from the mode. Configure the model, service endpoint, authentication, context, privacy, and reply language in **Settings → AI**.

Import `examples/ai/presets.json` for example Ollama, OpenAI, Gemini, DeepSeek, and Grok configurations. Local Ollama does not require an API key. Cloud services may send trace evidence to an external provider; use the anonymization and sensitive-trace settings when appropriate.

See AI.md for setup, privacy, model options, tools, troubleshooting, CLI testing, and evaluation details.

Export

BTFViewer provides the following export actions.

Output	Action
Annotated PNG or clipboard image	Open the **Snapshot editor**, add annotations if needed, then select **Save PNG** or **Copy**
Timeline SVG	**Save SVG**
Perfetto JSON	**Export Perfetto**
Selected BTF range	Place at least two cursors, then select **Save cursor range as BTF**
Statistics report	Open Statistics and select **Export HTML**
Trace comparison report	Open Compare and select **Export HTML**
Demonstration recording	Select **Record**, share the current tab, and stop recording to download a WebM file

Statistics and Trace Compare use a single **Export HTML** action. In the saved report, every table has a **CSV** button that downloads all of its rows matching the current **Search** and **Problems only** filters, in the current sort order (pagination is ignored). Clear those filters first to export the complete table. The **Anonymize** check box next to **Export HTML** replaces task names with stable `Task-N` aliases throughout the exported report.

Snapshot editor

Press `Ctrl+S` (or the activity-rail camera) to capture the current timeline view and open the annotation editor. It is the same design on Desktop and Web: a vertical tool rail, a properties panel that follows the selection, and a slim bar for history and zoom, so the canvas keeps the room and every control sits where the work is.

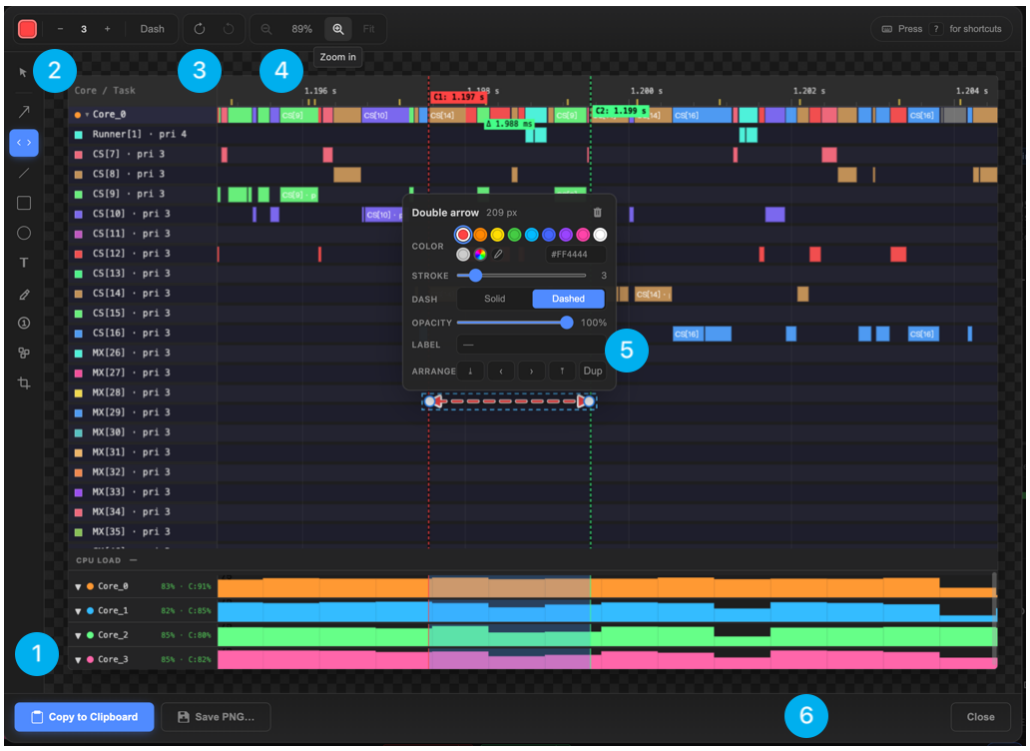

Figure 8: Snapshot editor

#	Area	What it does
1	**Tool rail**	**Select** first, then the annotation tools: arrow, double-arrow, line, rectangle, ellipse, text, highlighter, numbered badge, blur / redact, crop. Icon-only, one active tool; each has a single-key shortcut shown in its tooltip.
2	**Default style**	Colour, stroke width (– / +), and dash for the **next** shape drawn. Number keys 1–9, `0` also set the width. Editing an existing shape is done in the inspector (5).
3	**History**	Full undo / redo — draw, move, resize, restyle, delete and crop are all steps (`Ctrl+Z`, `Ctrl+Shift+Z`).

#	Area	What it does
4	**Zoom & pan**	Zoom-out and zoom-in (magnifier buttons), a percentage readout, and **Fit**. `Ctrl`+scroll zooms toward the pointer; hold Space and drag to pan. The view opens fitted.
5	**Floating inspector**	Appears on the selected shape and tracks it: colour (with recent colours and an image eyedropper), stroke, dash, opacity, label / text, font, z-order, duplicate, and a trash button to delete the shape.
6	**Footer**	**Copy to Clipboard**, **Save PNG…**, **Close**. When a crop is set, only the cropped region is exported.

`?` opens the keyboard-shortcuts panel. `Esc` peels back one step at a time — close a panel, deselect the shape, then return to the Select tool — it never closes the editor.

Desktop command line

The Desktop CLI uses the same analysis engine as the graphical application. Set `QT_QPA_PLATFORM=offscreen` when running without a display.

Command	Purpose
`info`	Print a trace summary
`report`	Generate a Statistics report
`compare`	Compare two traces
`analyze`	Check a trace against a baseline for CI
`ai-test`	Run AI evidence and validation tests
`migrations`	Export the migration table as CSV
`snapshot`	Save a timeline, migration, or metric image

Command	Purpose
`perfetto`	Export Chrome Trace JSON
`slice`	Save a selected timestamp range as a smaller BTF file

```
python builds/btf_viewer.py info trace.btf
python builds/btf_viewer.py report trace.btf -o report.html --format html
python builds/btf_viewer.py compare before.btf after.btf -o diff.html
python builds/btf_viewer.py analyze candidate.btf --baseline baseline.btf
↪  --fail-on-regression
python builds/btf_viewer.py snapshot trace.btf -o view.png --view timeline
python builds/btf_viewer.py perfetto trace.btf -o trace.json
python builds/btf_viewer.py slice trace.btf -o window.btf --lo 100000 --hi
↪  500000
```

Run `python builds/btf_viewer.py <command> -h` for all options.

Settings

Open **Settings** from the toolbar or press `Ctrl+,`.

Area	Options
Appearance	Theme, fonts, and colorblind-safe palette
Display	Panels, timeline overlays, CPU budget, and task deadlines
Layout	Label width, row height, zoom density, cursor limit, time precision, and chart sizes
AI	Enablement, context level, privacy, provider, model, authentication, and reply language

Desktop stores settings in `btf_viewer.rc` next to the viewer. Web stores them in browser `localStorage`. Changes are previewed immediately; select **OK** to save or **Cancel** to restore the previous values.

Keyboard and mouse

Common shortcuts

Key	Action
`Ctrl+O`	Open a file
`Ctrl+W`	Close the current tab
`Ctrl+Tab` / `Ctrl+Shift+Tab`	Move between tabs
`Ctrl++` / `Ctrl+-`	Zoom in, or zoom out until Fit Trace
`Ctrl+0` / `F`	Fit Trace (complete capture)
`Ctrl+R`	Fit Cursors (earliest–latest cursor span)
`Ctrl+F` / `F3` / `Shift+F3`	Find, next result, or previous result
`Ctrl+G`	Jump to a timestamp
`Ctrl+K`	Open the command palette (shortcuts, synonyms, recent/frequent; disabled actions show why)
`C` / `Shift+C`	Place a cursor or clear all cursors
`Ctrl+B`	Add a bookmark
`A`	Add an annotation
`Ctrl+S`	Open the Snapshot editor
`Ctrl+Shift+S`	Save SVG
`Ctrl+Shift+E`	Export Perfetto JSON
`?` / `F1`	Show the keyboard and mouse shortcuts reference

Mouse controls

Action	Result
Mouse wheel	Pan the timeline (vertical or horizontal depending on orientation)

Action	Result
Ctrl+wheel / pinch	Zoom the timeline
Left-drag the background	Pan
Middle-drag	Zoom to a selected range
Ctrl+left-drag	Measure the time between two points
Click the timeline	Place a cursor
Drag a cursor or mark	Move it
Right-click	Open the context menu

Build and test

Most users can ignore this section.

Task	Command
Build Desktop and Web	`make -C BTFViewer`
Build the Desktop package	`make -C BTFViewer bundle`
Build the Web application	`make -C BTFViewer web`
Run the guided demo	`make -C BTFViewer demo`
Run Desktop tests	`make -C BTFViewer test`
Run Web tests	`make -C BTFViewer test-web`
Run all tests	`make -C BTFViewer test-all`
Build documentation PDFs	`make -C BTFViewer doc`
Run from source	`python -m btf_viewer_pkg trace.btf` from `BTFViewer/`

See `TRACE_FORMAT.md` in the parent directory for the BTF field reference.

Contributors

Thanks to everyone who has contributed to this project.

Contributor	Contribution
DiogoRoseira	CPU Load Graph and metric-distribution charts